Tuned-GAE study: is ecbench's rho legitimate, and is its per-step cost fair?

Branch `claude/tuned-gae-20261020`, 2026-10-20. Host: Intel Xeon at 2.80 GHz, 4 logical CPUs, shared cloud VM, isolation L0.

Labels: **measured** is read from a harness record; **recorded derived** is computed by the repository's Rust tools and stored in a committed file; nothing is computed in Python. Source study directory: `research/tuned_gae_20261020/` (the README there has every table in full).

1. Answer

- **The GAE unit is untuned, and re-pricing it makes index calculus (IC) look worse.** A tuned batched affine addition, cross-checked addition by addition, is 3.16 to 4.93 times faster than the reference add that defines the GAE on prime curves (10 to 24 bits) and 8.68 to 21.69 times faster on binary curves (11 to 41 bits) (measured, 33 rows). Re-pricing the recorded Table B recipes raises IC's S by 2 to 19 percent and widens the gap to the matched rho on all 13 instances (6.34 to 6.49 at `bench-10bit`; 38.21 to 45.59 at `random-binary-n27`).
- **That is an accounting correction, not an improvement.** Counts are unchanged and the unit changed; under the success criterion it is recorded honestly and not reported as progress. It is proposed, not canonical: the legacy tables remain what `Calibration::pin`, `ecbench::band` and `full_price` read.
- **`rho.prime_fast` is a measured speed-up of the rho reference implementation.** At equal counted work (operations ratio 1.027, 95% interval 0.983 to 1.075, 1,024 pairs: no ranking on counted work) its median online wall is 0.2175 of `rho.negation`'s on four prime curves of 28 to 40 bits (L0). Its first configuration needed 2.05 times the operations (start-up cost), fixed in the tuned one.

2. Method

A tuned add (Montgomery u64 or pclmulqdq GF(2^n), 256-lane batched inversion) is timed beside `ic_boundary::calibrate_group` on `CountedGroup::add` and verified against it. Tuned tables `docs/ic/calibration_tuned.json` and `full_prices_tuned.json` (stamped `tuned-batched-affine-add/v1`) re-price each unit by its class: (a) counted group operation, unchanged; (b) native work on untuned field arithmetic, credited the measured field speedup F (clamped to at least 1); (c) already tuned or field-independent. `gae-reference reprice-ledger` reproduces every recorded legacy S within 1e-9, then prices the same counts under the tuned table. Three ecbench sessions compare `rho.prime_fast` with `rho.negation` on the same targets, alternating order.

3. Results

Fig. 1. Reference vs tuned affine addition by field size (gae-reference sweep, 33 rows, Xeon @ 2.80 GHz, L0 shared VM)

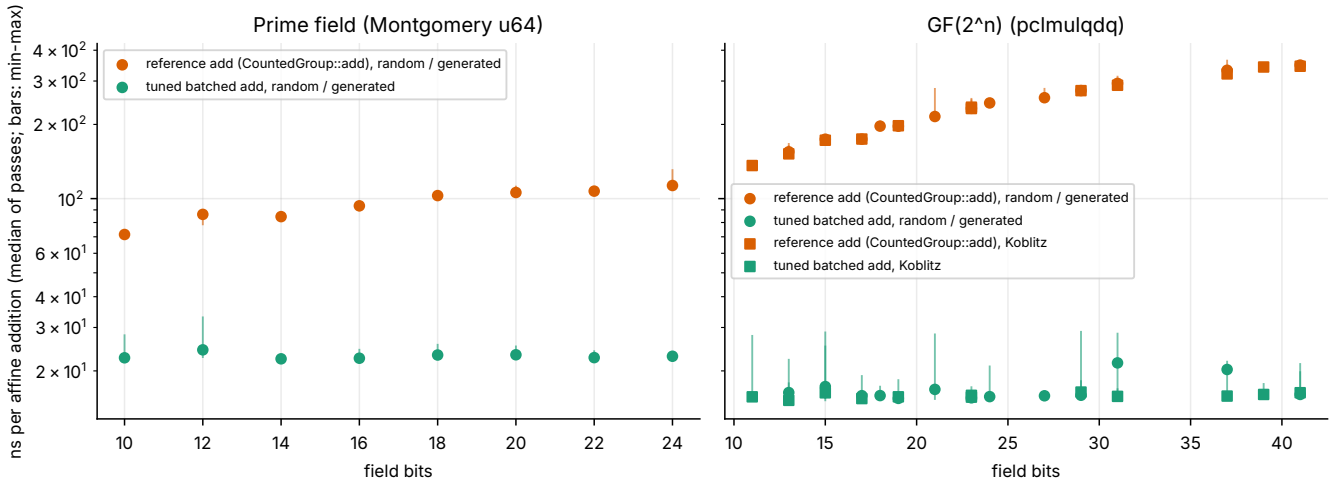


Figure 1: ns per affine addition, reference versus tuned, by field bits. Source `sweep/gae_reference.jsonl` (33 rows; tuned bars are min to max over 15 passes, reference bars the 3 recorded runs). Prime 3.16 to 4.93 times, binary 8.68 to 21.69 times.

Field-level (measured, `sweep/field.jsonl`): batched inversion is 3.63 to 7.22 times faster on prime and 12.31 to 48.07 times on binary; the modular row operation is 0.76 to 1.42 times, not faster in about half the rows.

Fig. 2. Table B best recipe per instance under the legacy and the tuned GAE unit (reprice/table_b.json; matched rho unchanged; bench = prime, bin = random binary)

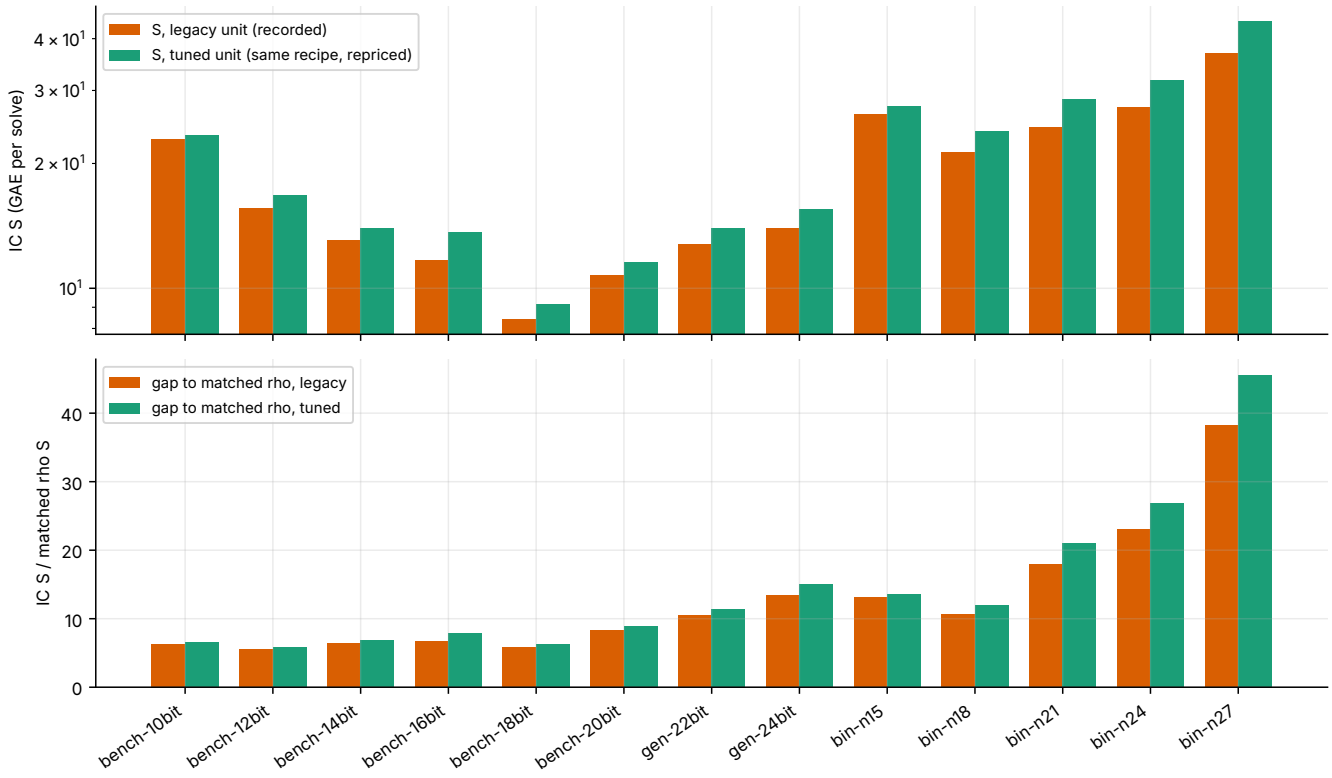


Figure 2: Table B best legacy recipe per instance, S and gap to the matched rho, legacy and tuned unit. Source `reprice/table_b.json` (recorded derived). The matched rho is unchanged.

Fig. 3. rho.negation vs rho.prime_fast, old and tuned config (ecbench table, all records verified, L0, --cpus none --allow-busy)

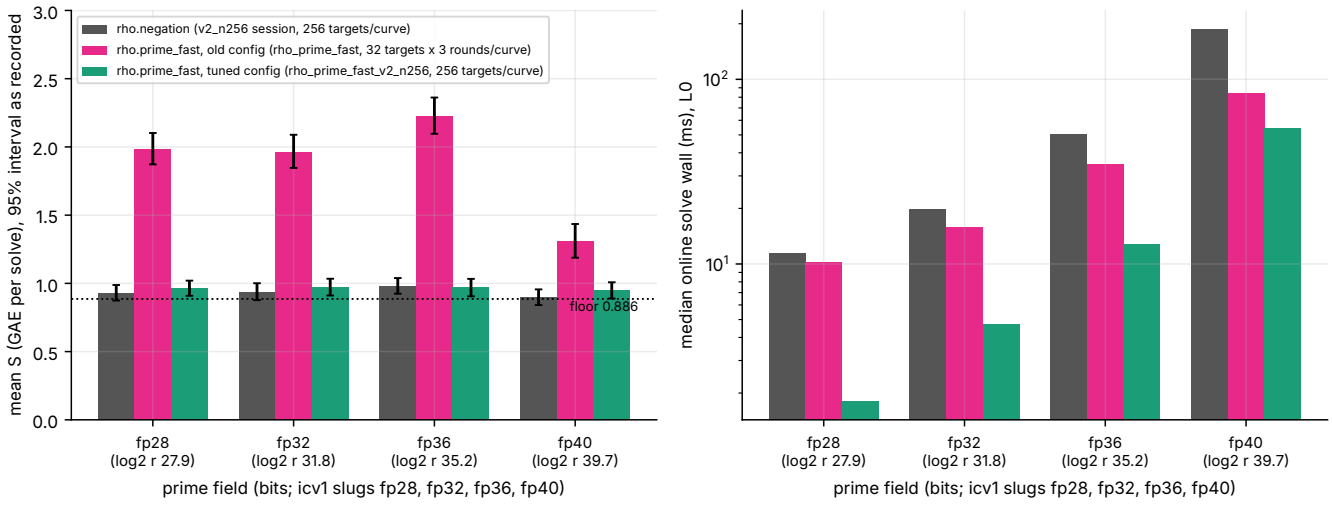


Figure 3: Mean S with the recorded 95% interval and median online wall per curve. Sessions ECBS1h3e27cfbb061d (old config, 32 targets x 3 rounds per curve) and ECBS1h522578c510c4 (tuned and `rho.negation`, 256 per curve). All records verified; L0.

session	config	targets / curve	ops fast / negation	median wall fast / negation
ECBS1h3e27cfbb061d	old	32 x 3 rounds	2.052 [1.897, 2.226]	0.821 [0.783, 0.855]
ECBS1h3da70394827f	tuned	32 x 3 rounds	1.097 [0.996, 1.210]	0.224 [0.210, 0.237]
ECBS1h522578c510c4	tuned	256	1.027 [0.983, 1.075]	0.2175 (interval degenerate, as recorded)

4. What counts under the success criterion

item	class	improvement?
Tuned GAE unit and Table B re-price	accounting	No. Counts unchanged; every gap widened (5.86 to 45.59 across Table B, the distance remaining).
`rho.prime_fast`	engineering, rho side	A measured speed-up of the rho implementation; no IC row moves; it raises the bar a wall comparison must clear.
IC S in tables A to C		Unchanged, legacy unit retained.

5. Finding, not fixed: malloc consolidation in the reference arm's timed window

About 9 ms of `rho.negation`'s timed solve window at small sizes is glibc `malloc_consolidate`, triggered by the 51,216-byte `fast_map(1024)` allocation after the JSON parse. With `GLIBC_TUNABLES=glibc.malloc.mxfast=0` `solve_wall_ns` fell from about 9.5 ms to about 0.17 ms on a 16-bit curve. No session record of this is committed; it is an ad hoc observation to reproduce (workload `specs/fixed_wall_probe.json`). It inflates the reference arm's wall at 28 to 32 bits, so the 0.2175 wall ratio overstates the algorithmic gain there; the 40-bit curve (187 against 54 ms, about 0.29) is least affected.

6. Sensitivity and limits

- **Cross-host assumption.** Tuned ratios multiply the legacy ones (a 2.10 GHz Xeon; the Koblitz GF(2^23) K_0 legacy row from an Apple M4 Pro under load) by this host's reference-over-tuned ratio and F. This host's own per-unit ratios drift from the legacy ones by a median of 1.77 (Legendre), 1.78 (sqrt), 1.60 (word XOR), 1.52 (Artin-Schreier), 1.13 (row operation) and 0.77 to 0.99 (canonicalisation, inversion, Frobenius, doubling). The sign of the change is robust because the addition speedup (3 to 22 times) dominates; its size (2 to 19 percent) is not host-independent.
- The old-config session and the sweep overlapped in time on the same 4-CPU VM (16:30:46 to 16:32:30 and 16:31:29 to 16:32:15 UTC); no contention telemetry for the sweep.
- L0 only; wall figures are descriptive. The section 5a 0, 0.1, 1 GAE band was not re-run; no ranking between arms is made.
- Prime 28 to 40 bits for `rho.prime_fast`; Binary and Koblitz keep `rho.signed_frobenius_strong`, not re-timed.

7. Open follow-ups

1. Switch plan (`SWITCH_PLAN.md`): `gae_definition` on every record, table chosen from the record, re-pin on the isolated host.
2. A prime `vs_rho` claim branch (`ecbench claim build` has none; the wall reference is named in AGENTS.md only).
3. Check the binary and Koblitz strong rho for the same allocator cost.
4. Harden the reference arm's timed window and commit a record for the malloc finding.
5. Re-run the n256 session at L2 or higher for a real wall interval.